

FUNDAMENTOS JAVA.

Aprende todo lo relacionado con los fundamentos de java de forma fácil y sencilla.

Kevin Lopez – Instructor.

Nunca es tarde
para aprender.

Sobre el libro.

Este libro es una guía completa y accesible diseñada para introducir a los lectores en el mundo del lenguaje de programación Java. Este libro cubre los aspectos esenciales del lenguaje, comenzando con un recorrido por su historia, destacando su evolución y características que lo han convertido en uno de los lenguajes más populares del mundo. A medida que los lectores avanzan, explorarán los conceptos básicos de la programación, la estructura del lenguaje, el manejo de control de flujo, y culminarán con una profunda introducción a la Programación Orientada a Objetos (POO), incluyendo clases, objetos y más. Ideal para principiantes, este libro proporciona una base sólida para dominar Java y sentar las bases para futuros proyectos de desarrollo."

Este enfoque equilibrado ofrece tanto teoría como aplicaciones prácticas para que los lectores comprendan y apliquen lo aprendido.

Índice

Contenido

¿Qué es java?	4
Historia de java.....	4
Ventajas y desventajas de usar java	5
¿Por qué usar java?	6
Variables en java	7
Comentarios en java	9
¿Qué es un comentario?	9
Comentarios de una sola línea	9
Comentario de múltiple línea o bloque	9
Comentarios javadoc	10
Operaciones aritméticas	10
Toma de datos en Java	11
Scanner.....	12
Biblioteca JOption.....	14
Condicionales en Java	15
Condicional IF-ELSE	15
Sentencia Switch	17
Sentencia Else-If	20
Arreglos o arrays en Java	20
Listas.....	21
Ciclos y Bucles en java	23
Bucle While	23
Bucle DO-WHILE.....	24
Bucle FOR	26
Funciones en java	27
Parámetros en funciones.....	28
Programación orientada a Objetos	29
Objetos en POO java.....	30
Ejercicios de práctica	34
Próximos pasos para convertirte en un desarrollador profesional	36
.....	37

Recursos adicionales.	37
Aprende más sobre Java:.....	37
Sigue aprendiendo:.....	37

¿Qué es java?

Java es un lenguaje de programación orientado a objetos, diseñado para ser independiente de la plataforma y que permite a los desarrolladores escribir software que pueda ejecutarse en cualquier sistema operativo. Creado por Sun Microsystems en 1995 y actualmente mantenido por Oracle, Java se destaca por su robustez, seguridad y escalabilidad. Es ampliamente utilizado en una variedad de aplicaciones, desde aplicaciones móviles y web hasta sistemas empresariales complejos, debido a su capacidad para gestionar grandes volúmenes de datos y transacciones de manera eficiente. Java también cuenta con una vasta comunidad y un ecosistema rico de bibliotecas y frameworks que simplifican el desarrollo de software.

Historia de java.

Java es un lenguaje de programación desarrollado por Sun Microsystems (ahora parte de Oracle Corporation) en 1995. Su historia comienza a principios de los años 90, cuando un equipo liderado por James Gosling, conocido como el "padre de Java", comenzó a trabajar en un proyecto llamado "**Green Project**". Este proyecto tenía como objetivo crear un lenguaje de programación que pudiera ser utilizado en dispositivos electrónicos como televisores, decodificadores y otros equipos electrónicos domésticos. Inicialmente, el lenguaje fue llamado **Oak**, pero este nombre ya estaba registrado, por lo que se cambió a **Java**.

El objetivo principal de Java era desarrollar un lenguaje de programación que fuera:

1. **Independiente de la plataforma:** Java fue diseñado para ser independiente del sistema operativo o del hardware, siguiendo el principio de "escribir una vez, ejecutar en cualquier lugar" (*write once, run anywhere*). Esto fue posible gracias a la máquina virtual de Java (**JVM**), que permite que los programas escritos en Java se ejecuten en cualquier dispositivo que tenga una JVM instalada, sin necesidad de modificarlos.
2. **Orientado a objetos:** Java adopta la Programación Orientada a Objetos (POO), lo que facilita el desarrollo de aplicaciones modulares, reutilizables y fáciles de mantener.
3. **Seguro y robusto:** Java fue diseñado con características de seguridad integradas, como la gestión automática de memoria a través del recolector de basura y la eliminación del uso directo de punteros, lo que reduce los errores y las vulnerabilidades de seguridad.
4. **Multihilo:** Desde el principio, Java soportó la programación multihilo, lo que permite que los programas realicen varias tareas simultáneamente, mejorando el rendimiento en aplicaciones concurrentes.

Evolución del Lenguaje:

- **Java 1.0 (1996):** Fue la primera versión pública de Java, lanzada como "Java 1.0". En este momento, Java se utilizaba principalmente para aplicaciones web a través de "applets", pequeños programas que se podían ejecutar en navegadores con soporte de JVM. Aunque el uso de applets se ha reducido, marcó el inicio de Java en la web.
- **Java 2 (1998):** Introdujo la "Plataforma Java 2", que incluía nuevas características y una división en tres ediciones: **Java Standard Edition (Java SE)** para aplicaciones de escritorio, **Java Enterprise Edition (Java EE)** para aplicaciones empresariales, y **Java Micro Edition (Java ME)** para dispositivos móviles y embebidos.

- **Java 5 (2004):** Esta versión fue un hito importante al introducir características modernas como **genéricos**, **autoboxing**, **tipos enumerados** y la mejora en el manejo de hilos.
- **Java 8 (2014):** Una de las versiones más importantes de la historia de Java, ya que introdujo **expresiones lambda**, lo que permitió un estilo de programación más funcional, y la **API de Streams**, que facilitó el procesamiento de grandes conjuntos de datos.
- **Java 9, 10, 11 y posteriores:** A partir de Java 9 (2017), se introdujo un nuevo ciclo de lanzamientos rápidos, con una nueva versión cada seis meses. Las versiones recientes han incluido mejoras en el rendimiento, la modularidad con el **sistema de módulos** introducido en Java 9, y soporte extendido a larga duración (LTS) en Java 11.

Actualidad de Java.

En la actualidad, Java es uno de los lenguajes de programación más populares y ampliamente utilizados en el mundo, con aplicaciones que abarcan desde sistemas empresariales hasta aplicaciones móviles (a través de Android), servicios en la nube, e Internet de las cosas (IoT). La comunidad Java sigue siendo vibrante, con continuas mejoras en el lenguaje y en la JVM para mantener su relevancia en el ecosistema de desarrollo de software.

Java se ha consolidado como una tecnología clave en el mundo de la programación gracias a su robustez, versatilidad y capacidad para evolucionar con el tiempo.

Ventajas y desventajas de usar java.

Ventajas de Java:

1. **Independencia de plataforma:** Java es famoso por su lema "Escribe una vez, ejecuta en cualquier lugar" (WORA). Gracias a la Máquina Virtual de Java (JVM), el código puede ejecutarse en cualquier sistema operativo que tenga instalada una JVM, sin necesidad de modificaciones.
2. **Orientado a objetos:** Java promueve el diseño modular y reutilizable del software, lo que facilita la creación de aplicaciones mantenibles y escalables.
3. **Gran comunidad y soporte:** Java cuenta con una comunidad global de desarrolladores y una amplia gama de bibliotecas y frameworks, lo que simplifica la resolución de problemas y el desarrollo de aplicaciones.
4. **Seguridad:** Java tiene varias características de seguridad integradas, como la gestión automática de la memoria, control de acceso y la capacidad de ejecutar código en entornos controlados.
5. **Multithreading:** Java facilita la programación concurrente, permitiendo que varias partes de una aplicación se ejecuten en paralelo, mejorando el rendimiento en aplicaciones complejas.
6. **Uso en aplicaciones empresariales y grandes sistemas:** Java es una opción popular en el desarrollo de sistemas empresariales robustos, ya que es altamente escalable y confiable.

Desventajas de Java:

1. **Rendimiento:** Aunque Java ha mejorado significativamente en términos de rendimiento, no es tan rápido como los lenguajes de bajo nivel como C o C++. La JVM introduce una capa de abstracción que puede afectar la velocidad.
2. **Consumo de memoria:** Las aplicaciones Java tienden a consumir más memoria en comparación con otras alternativas debido a la gestión automática de memoria (garbage collection).
3. **Curva de aprendizaje:** A pesar de ser un lenguaje bien documentado, Java puede ser complejo para principiantes debido a su naturaleza orientada a objetos y la cantidad de herramientas y configuraciones que debe conocer el desarrollador.
4. **Verbosidad:** Java requiere una gran cantidad de código para tareas relativamente simples, lo que puede hacer que los programas sean más largos y difíciles de leer en comparación con otros lenguajes como Python.
5. **Actualizaciones frecuentes:** Aunque las actualizaciones constantes mejoran el lenguaje, también pueden causar problemas de compatibilidad y obligar a los desarrolladores a estar al día con las versiones más recientes.

¿Por qué usar java?

Ya sabemos las ventajas y desventajas de usar lo que es java, pero aun sabiendo esto nace la pregunta **¿Por qué usar java?** Aquí te dejo 5 razones para usar java en tus futuros proyectos:

5 razones para Usar Java:

1. **Independencia de plataforma: "Escribe una vez, ejecuta en cualquier lugar":** Java es uno de los pocos lenguajes de programación que permite a los desarrolladores escribir su código una vez y ejecutarlo en cualquier sistema operativo, sin necesidad de modificar nada. Gracias a la Máquina Virtual de Java (JVM), el código puede funcionar en Linux, Windows, macOS y muchas otras plataformas. Esta flexibilidad es invaluable en el mundo del desarrollo moderno, donde las aplicaciones deben ser accesibles desde cualquier dispositivo.
2. **Fuerte enfoque en la programación orientada a objetos:** En Java, todo está orientado a objetos. Esto significa que los programadores pueden organizar su código en estructuras lógicas y reutilizables, como clases y objetos, lo que no solo hace que el software sea más fácil de mantener, sino también más modular y escalable. La programación orientada a objetos de Java fomenta la creación de aplicaciones robustas, que se pueden expandir y actualizar con mayor facilidad a medida que crecen.
3. **Rendimiento confiable y mejora continua:** Aunque Java no siempre ha sido reconocido por su velocidad en comparación con lenguajes de bajo nivel, como C++, las mejoras en la JVM y la gestión de memoria han optimizado su rendimiento de manera considerable. En aplicaciones de gran escala y empresariales, Java ha demostrado ser altamente eficiente, manejando grandes volúmenes de datos y operaciones sin comprometer la estabilidad.

4. **Extenso ecosistema de bibliotecas y frameworks:**
El ecosistema de Java es inmenso. Desde frameworks populares como Spring para el desarrollo empresarial, hasta herramientas de prueba como JUnit y bibliotecas gráficas como JavaFX, los desarrolladores tienen a su disposición una vasta colección de herramientas que facilitan casi cualquier tipo de proyecto. Esto acelera el desarrollo y reduce el tiempo necesario para crear soluciones complejas, gracias a las contribuciones de la gran comunidad global de desarrolladores de Java.
5. **Seguridad y gestión de memoria automatizada:**
Una de las características que distingue a Java es su fuerte enfoque en la seguridad. La gestión automática de la memoria mediante el *Garbage Collector* ayuda a prevenir fugas de memoria, uno de los problemas más comunes en el desarrollo de software. Además, Java ofrece un entorno seguro y controlado, protegiendo el sistema del código malicioso a través de controles de acceso y mecanismos de seguridad que son vitales en aplicaciones empresariales o financieras.

Variables en java.

Cuando iniciamos un lenguaje de programación, se nos vienen muchas preguntas a la cabeza como: ¿Qué debo hacer para empezar con el lenguaje?, ¿Debo empezar con lo más difícil?, estas preguntas pueden dejarte muy confundido y asustado por empezar con un lenguaje, como lo es JAVA, pero déjame decirte que es todo lo contrario, todo lenguaje es fácil de aprender.

Para ello, primero veremos qué son las variables, ya que es un concepto importante al momento de iniciar con un lenguaje de programación.

En Java, una **variable** es un espacio de almacenamiento en memoria reservado para almacenar un valor que puede cambiar durante la ejecución del programa. Cada variable tiene un **tipo de dato** asociado, lo que determina qué tipo de valor puede almacenar (números, caracteres, booleanos, etc.). Las variables en Java deben ser declaradas antes de ser utilizadas, y pueden tomar diferentes valores a lo largo del ciclo de vida del programa.

Tipos de Variables en Java:

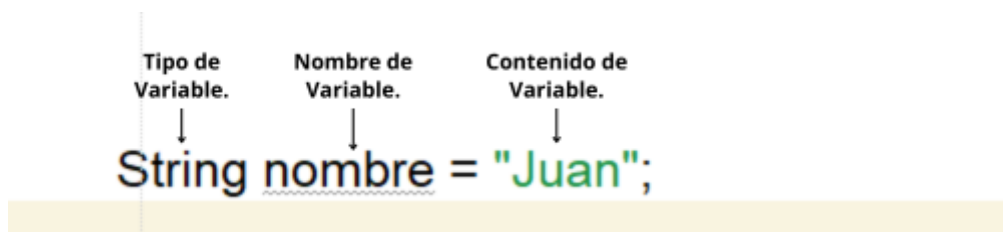
Java tiene dos categorías principales de variables:

1. **Variables Primitivas:** Son las más básicas y contienen directamente los datos. Los tipos primitivos son:
 - **byte:** 8 bits, almacena números enteros en un rango de -128 a 127.
 - **short:** 16 bits, almacena enteros más grandes, de -32,768 a 32,767.
 - **int:** 32 bits, el tipo más común para enteros, con un rango de -2^{31} a $2^{31} - 1$.
 - **long:** 64 bits, usado para enteros más grandes, de -2^{63} a $2^{63} - 1$.
 - **float:** 32 bits, almacena números en coma flotante (números decimales).
 - **double:** 64 bits, para números decimales de mayor precisión.

- **char**: 16 bits, representa un solo carácter Unicode.
 - **boolean**: 1 bit, almacena solo dos posibles valores: true o false.
2. **Variables de Referencia**: Estas variables no almacenan directamente el valor, sino que guardan la dirección de memoria donde se encuentra el objeto. Los tipos de referencia incluyen:
- **Objetos**: Instancias de clases definidas por el usuario.
 - **Arrays**: Estructuras que almacenan múltiples valores del mismo tipo.

¿Cómo declaramos una variable?

Para poder declarar una variable debemos saber la sintaxis para declararla, la estructura es la siguiente:



Primero, declaramos el tipo de variable, siempre debemos declarar el tipo de variable, ya que Java no reconocerá qué tipo de variable quieres declarar. Luego asignamos un identificador a la variable o un nombre para identificarla, para posteriormente declarar su contenido siendo este igual al tipo de dato que has declarado la variable.

Siguiendo esta lógica puedes declarar muchas variables de diferentes tipos, como variables de tipo entero, cadena de texto, decimales, booleanas, arreglos e inclusive Objetos.

```
public static void main(String[] args) {
    // Tipos de datos primitivos
    byte edad = 25;      // byte: 8 bits
    short salario = 15000; // short: 16 bits
    int numero = 123456789; // int: 32 bits
    long distancia = 9876543210L; // long: 64 bits (agregar 'L' al final)

    float pi = 3.1416f; // float: 32 bits (agregar 'f' al final)
    double gravedad = 9.81; // double: 64 bits

    char letra = 'A'; // char: 16 bits, almacena un carácter
    boolean esVerdad = true; // boolean: 1 bit, almacena true o false
    // Variable de referencia: String (una clase)
```

Entendiendo esto puedes escribir diferentes variables las cuales servirán para almacenar datos que necesites dentro de tu proyecto usando Java.

Comentarios en java.

¿Qué es un comentario?

Los **comentarios en Java** son fragmentos de texto dentro del código que el compilador ignora por completo. Se utilizan para anotar, explicar o documentar el código, facilitando su comprensión tanto para el propio programador como para otros desarrolladores que lo revisen en el futuro. Aunque no afectan la ejecución del programa, los comentarios son fundamentales para mantener un código legible y fácil de mantener.

Tenemos diferentes tipos de comentarios que puedes colocar en Java los cuales son los siguientes:

- **Comentarios de una sola línea.**
- **Comentario de múltiples líneas**
- **Comentarios javadoc**

Comentarios de una sola línea.

Se utilizan para hacer anotaciones breves en una sola línea. La forma de declarar comentarios de una sola línea es la siguiente:

```
// Esto es un comentario de una sola línea  
int edad = 25; // Definimos la variable 'edad'
```

Solo se utiliza solo dos plecas para definir un comentario de una sola línea, el editor con el que trabajes detectara automáticamente que este es un comentario.

Comentario de múltiple línea o bloque.

Se utilizan para escribir comentarios largos o describir bloques completos de código. La forma de declarar comentarios Múltiples o bloque es la siguiente:

```
/*  
 * Este es un comentario de varias líneas  
 * Se puede usar para describir el código o explicar la lógica de una sección.  
 */  
int salario = 15000;
```

Comentarios javadoc.

Estos comentarios especiales se utilizan para generar documentación automática para el código. Son ideales para documentar clases, métodos y atributos. Herramientas como Javadoc pueden leer estos comentarios y generar documentación en formato HTML. La forma de declarar comentarios Múltiples o bloque es la siguiente:

```
/**
 * Método que calcula la suma de dos números.
 * @param a primer número
 * @param b segundo número
 * @return suma de a y b
 */
public int sumar(int a, int b) {
    return a + b;
}
```

Operaciones aritméticas.

Las **operaciones aritméticas en Java** son operaciones matemáticas básicas que se realizan entre dos o más valores numéricos (pueden ser variables o constantes). Estas operaciones son fundamentales en cualquier lenguaje de programación y permiten realizar cálculos con números enteros, decimales, y otras estructuras numéricas.

En Java, se utilizan **operadores aritméticos** que siguen las reglas estándar de la aritmética. Las operaciones más comunes son la suma, resta, multiplicación, división y el cálculo del residuo o módulo.

La forma de declarar operaciones aritméticas en java es casi la misma que en la vida real, te dejare los signos para cada operación aritmética que puedes declarar en java.

Suma (+): Se utiliza para sumar dos valores.

- Ejemplo: a + b

Resta (-): Se utiliza para restar un valor de otro.

- Ejemplo: a - b

Multiplicación (*): Se utiliza para multiplicar dos valores.

- Ejemplo: a * b

División (/): Se utiliza para dividir un valor por otro. En la división de enteros, el resultado es también un entero (se descarta la parte decimal), mientras que en la división de números decimales se obtiene un resultado con decimales.

- Ejemplo: a / b

Módulo (%): Devuelve el residuo de la división entre dos números. Es útil para saber si un número es divisible por otro.

- Ejemplo: $a \% b$

Estos son todos los signos que puedes declarar en java, en código se vería de la siguiente manera:

```
// Declaración de variables
int a = 10;
int b = 3;

// Suma
int suma = a + b; // 10 + 3 = 13
System.out.println("Suma: " + suma);

// Resta
int resta = a - b; // 10 - 3 = 7
System.out.println("Resta: " + resta);

// Multiplicación
int multiplicacion = a * b; // 10 * 3 = 30
System.out.println("Multiplicación: " + multiplicacion);

// División
int division = a / b; // 10 / 3 = 3 (división de enteros)
System.out.println("División (entera): " + division);

// Módulo
int modulo = a % b; // 10 % 3 = 1 (residuo de la división)
System.out.println("Módulo: " + modulo);
```

En cada una de las operaciones ocupamos los diferentes signos que representan cada una de las operaciones aritméticas estos usando un println regresan el resultado de la operación para ser mostrada en consola.

Este conjunto de operaciones es útil para realizar cálculos matemáticos en cualquier aplicación, desde simples calculadoras hasta sistemas complejos de procesamiento de datos.

Toma de datos en Java.

Ya hemos aprendido a como declarar las variables como sus comentarios incluyendo las diferentes operaciones aritméticas dentro de java, ahora aprenderemos a como pedirle datos al usuario.

Cuando hablamos de toma de datos nos referimos al proceso mediante el cual un programa recibe información del usuario para procesarla o utilizarla en la lógica del sistema. Esto se puede hacer de diferentes maneras, dependiendo del tipo de entrada y del entorno en el que se esté ejecutando el programa.

Hay diferentes formas en las cuales puedes pedir datos al usuario en java, pero te mostrare las dos más utilizadas para pedir datos al usuario.

Scanner.

La clase Scanner es la forma más común y sencilla para leer datos desde la consola. Permite leer diferentes tipos de datos como enteros, decimales, cadenas, y otros, de una manera fácil.

Primero para usar este método para pedir datos, debemos declarar un objeto el cual será de tipo Scanner, cuando la declaremos java detectara que necesitamos importar la librería respectiva para que el método funcione correctamente entonces importara lo necesario.

```
package com.achirou;
// Crear un objeto Scanner para leer desde la entrada estándar (teclado)
Scanner scanner = new Scanner(System.in);
```

Aquí importamos el paquete.

```
import java.util.Scanner;
```

Luego debemos dar un mensaje a el usuario que le indique que va a ingresar un dato al programa entonces le mandamos un mensaje en consola, luego crearemos una variable de cualquier tipo, donde le especificaremos que el scanner detectara un tipo de dato usando su método **next**, en el caso de que el tipo de dato será entero o diferente a String le especificaremos el tipo de dato después del next (aparte de haber especificado el tipo de dato en la variable.

```
// Leer un número entero
System.out.print("Ingresa un número entero: ");
int numero = scanner.nextInt();
```

Nosotros en este ejemplo pediremos 3 datos como veras a continuación.

```
// Leer un número entero
System.out.print("Ingresa un número entero: ");
int numero = scanner.nextInt();

// Leer una cadena de texto
System.out.print("Ingresa tu nombre: ");
String nombre = scanner.next();

// Leer un número decimal
System.out.print("Ingresa un número decimal: ");
double decimal = scanner.nextDouble();
```

Con estas declaraciones el scanner detectara automáticamente que pediremos datos al usuario. Ahora solo falta mostrarlos en pantalla, para ello usaremos un println y es importante que cerremos el scanner para que cierre todo el proceso.

```
// Mostrar los datos ingresados
System.out.println("Número entero: " + numero);
System.out.println("Nombre: " + nombre);
System.out.println("Número decimal: " + decimal);

// Cerrar el scanner
scanner.close();
```

Al ejecutarlo te pedirá los tres diferentes tipos de datos y te dará una respuesta como esta:

```
] --- exec:3.1.0:exec (default-cli) @ BOOK ---
Ingresa un número entero: 2
Ingresa tu nombre: Kevin
Ingresa un número decimal: 2.20
Número entero: 2
Nombre: Kevin
- Número decimal: 2.2
```

Biblioteca JOptionPane.

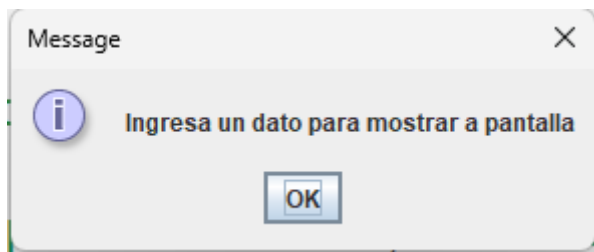
JOptionPane es una clase clave dentro de la biblioteca Java Swing que facilita la creación de cuadros de diálogo interactivos con el usuario. A través de sus métodos, permite mostrar mensajes informativos, solicitar datos o plantear preguntas con opciones predefinidas como "Sí", "No" y "Cancelar", entre otras. Esta clase simplifica la interacción entre la interfaz gráfica y el usuario, haciendo más accesible la implementación de diálogos en aplicaciones Java.

Cuando se utiliza **JOptionPane** para mostrar un mensaje en pantalla, es necesario importar su correspondiente clase. Los entornos de desarrollo integrados (IDE), como NetBeans o IntelliJ IDEA, generalmente sugieren o incluso añaden de forma automática las importaciones necesarias al escribir el código, facilitando el proceso para el desarrollador.

Diagrama de flujo para la implementación de un mensaje:

```
graph TD
    A[Iniciación de librería.] --> B[Selección de output al usuario.]
    B --> C[Mensaje.]
    C --> D[JOptionPane.showMessageDialog(null, "Ingresa un dato para mostrar a pantalla");]
```

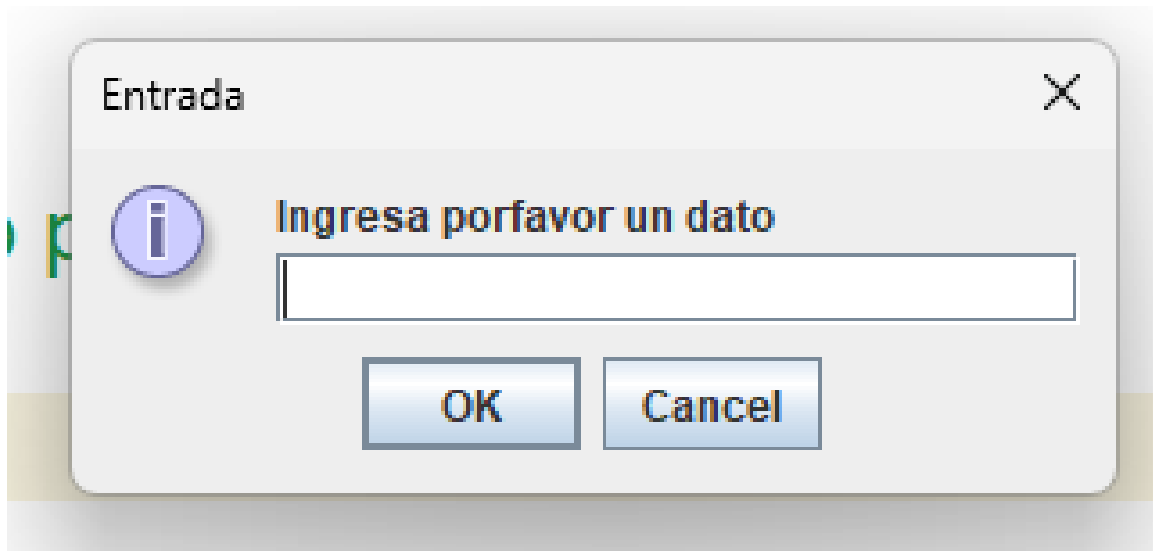
Como podemos ver en el ejemplo primero iniciamos el componente de la librería, luego seleccionamos el tipo de output que se le mostrara al usuario para posteriormente mandar el mensaje al usuario que vera en pantalla.



Aparte de poder mostrar datos también podemos pedir los datos al usuario, para ello en el output escogemos en lugar de un **showMessage** escogemos un **inputMessageDialog**

```
String dato = JOptionPane.showInputDialog(null, "Ingresa porfavor un dato", "Entrada", JOptionPane.INFORMATION_MESSAGE);
```

Aparte de cambiar el tipo de output este lo guardaremos en una variable ya que la información que reciba debe ser guardado en una variable, al colocarlo nos mostrara lo siguiente.



Donde podremos ver que ahora automáticamente nos añade un campo para introducir texto, de esta manera podemos pedir datos al usuario utilizando esta biblioteca de Java.

Condicionales en Java.

En Java, las estructuras condicionales permiten que un programa tome decisiones basadas en el cumplimiento de ciertas condiciones. Estas estructuras controlan el flujo de ejecución al evaluar expresiones booleanas (que resultan en verdadero o falso), lo que determina qué bloque de código se ejecutará.

La instrucción `if` es la más básica y permite ejecutar un bloque de código si una condición específica es verdadera. En combinación con `else`, se pueden definir rutas alternativas de ejecución para cuando la condición sea falsa. Además, Java ofrece la instrucción `else if` para evaluar múltiples condiciones secuencialmente.

Otra estructura útil es el `switch`, que selecciona una de muchas ramas posibles de ejecución según el valor de una expresión. Esta es ideal cuando se manejan múltiples valores posibles para una misma variable, simplificando el código en comparación con múltiples `if-else`.

Estas estructuras son fundamentales para construir programas que puedan reaccionar dinámicamente a diferentes situaciones o entradas.

Para entender mejor lo que son las condicionales vamos a desglosarlas poco a poco.

Condicional IF-ELSE.

La estructura `if-else` en Java es una de las más utilizadas para controlar el flujo de un programa. Permite ejecutar un bloque de código cuando una condición es verdadera y, opcionalmente, otro bloque cuando la condición es falsa.

El bloque `if` evalúa una expresión booleana; si el resultado es `true`, el código dentro de ese bloque se ejecuta. Si es `false`, el programa salta al bloque `else`, donde se define la alternativa a ejecutar. Esta estructura es fundamental para la toma de decisiones en tiempo de ejecución, permitiendo

que el programa reaccione a diferentes situaciones o datos de entrada de manera flexible y eficiente.

La estructura de dicha condicional es la siguiente:

```
Sentencia IF.  
if( /*condicional*/){  
    Condicional.  
    //Argumento    Argumento.  
  
}else{    Respuesta opuesta al condicional.  
    //Argumento    Argumento.  
}
```

Veamos un ejemplo para que te quede más claro:

Primero declaremos dos variables, estas se llamarán **numero1** y **numero2** como lo vemos en la siguiente imagen:

```
public static void main(String[] args) {  
  
    //Declaración de variable  
    int numero1 = 5;  
    int numero2 = 3;
```

Luego de declarar ambas variables procedamos a declarar la sentencia if- else donde primero declaramos la sentencia IF y luego su respectiva condicional, en este caso le diremos que nos indique si el numero1 es mayor al numero2.

```
if(numero1 > numero2 ){
```

Seguido de unas llaves nosotros pondremos el argumento o código que se ejecutara si la condición es verdadera, en nuestro caso colocaremos un mensaje personalizado.

```
if(numero1 > numero2 ){  
    System.out.println(" El numero 1 : " + numero1 + " Es mayor al numero 2: " + numero2);
```

Luego de colocar la condicional debemos indicar su contraparte, si la condicional no es correcta ósea numero1 no es mayor a numero2 entonces ejecutara la contraparte Else:

```

}else{

System.out.println(" El numero 2 : " + numero2 + " Es mayor al numero 1: " + numero1);

}

```

Esta estructura en código completo quedaría de la siguiente manera:

```

//Declaración de variable
int numero1 = 5;
int numero2 = 3;

if(numero1 > numero2 ){

    System.out.println(" El numero 1 : " + numero1 + " Es mayor al numero 2: " + numero2);

}else{

    System.out.println(" El numero 2 : " + numero2 + " Es mayor al numero 1: " + numero1);

}

```

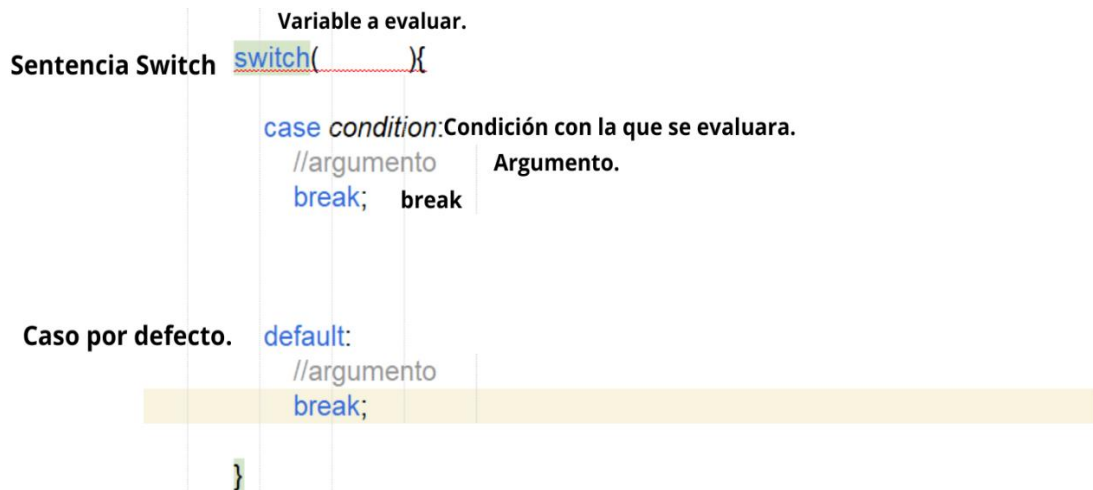
De esta manera podemos utilizar la sentencia If-Else dentro del entorno de java para la toma de decisiones en nuestro código y establecer un mejor control sobre él y el rumbo que tomara en cada situación.

Sentencia Switch.

La sentencia switch en Java es una estructura de control que simplifica la toma de decisiones cuando se deben comparar múltiples posibles valores de una misma variable. En lugar de escribir una serie de condicionales if-else, el **switch** permite seleccionar entre varias ramas de ejecución, dependiendo del valor de una expresión.

El funcionamiento es sencillo: la variable evaluada se compara con diferentes valores definidos en bloques llamados **casos (case)**. Si alguno de estos casos coincide con el valor de la variable, el código correspondiente se ejecuta. Opcionalmente, se puede incluir un bloque **default** para manejar cualquier situación que no coincida con los casos especificados. Esta estructura es útil cuando se manejan múltiples opciones posibles de forma clara y concisa, mejorando la legibilidad del código en comparación con una cadena de condicionales.

Veamos su estructura para que lo comprendas de una mejor manera:



Para tener un mejor concepto de utilización procedamos a hacer un ejemplo práctico:

Primero creamos una variable que va a ser evaluada en el case como en la siguiente imagen:

```
int dia = 3; // Día de la semana (1 = Lunes, 2 = Martes, ..., 7 = Domingo)
```

Luego definimos nuestra sentencia switch y en su condicional colocamos la variable que será evaluada en los distintos tipos de cases:

```
switch (dia) {
```

Una vez ya definimos la variable en la condicional ahora crearemos diferentes tipos de case que evalúen el contenido de la variable colocando el termino **case** seguido del valor a evaluar, esto puede ser cualquier tipo de dato:

```
switch (dia) {  
    case 1:  
        System.out.println("Lunes");  
        break;  
    case 2:  
        System.out.println("Martes");  
        break;  
    case 3:  
        System.out.println("Miércoles");  
        break;  
    case 4:  
        System.out.println("Jueves");  
        break;  
    case 5:  
        System.out.println("Viernes");  
        break;  
    case 6:  
        System.out.println("Sábado");  
        break;  
    case 7:  
        System.out.println("Domingo");  
        break;  
}
```

En cada uno de los casos colocaremos lo que es un break que nos ayudara a que cuando el valor de la variable coincida con alguno de los casos entonces el código se detenga ahí y no siga ejecutando los demás case.

Por último puedes agregar un caso por defecto, esto es opcional ya que no muchos lo ocupan.

```
default:  
    System.out.println("Día inválido");  
    break;
```

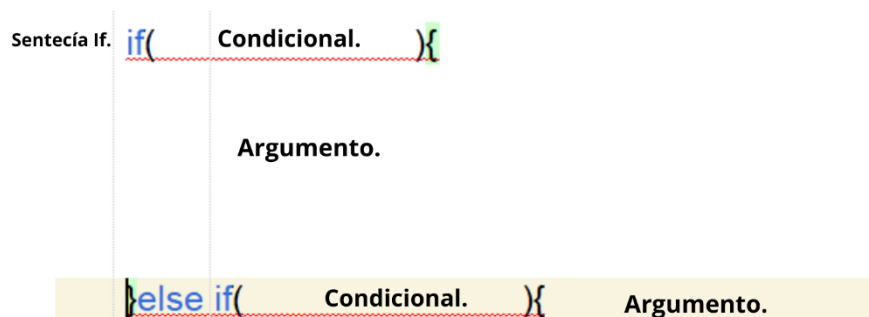
Con esto podemos manejar también decisiones dentro de lo que sería tu código y que tome un rumbo en cada situación que se pueda presentar según su finalidad.

Sentencia Else-If.

La estructura else-if en Java es una extensión de la sentencia if-else que permite evaluar múltiples condiciones secuencialmente. Después de un bloque if, se pueden agregar uno o más bloques else if para verificar condiciones adicionales si la condición inicial resulta falsa. Si alguna de las condiciones en los bloques else if es verdadera, se ejecuta el código asociado a esa condición y el flujo del programa continúa.

Esta estructura es especialmente útil cuando se requiere tomar decisiones entre varias opciones mutuamente excluyentes, ya que permite evaluar diferentes casos sin necesidad de anidar múltiples sentencias if. Si ninguna de las condiciones es verdadera, se puede agregar un bloque final else para manejar el caso por defecto. Esto hace que el código sea más organizado y claro cuando se deben evaluar varias posibilidades.

La Estructura para esta condicional es la siguiente:



Siguiendo esta estructura puedes establecer diversos **ELSE-IF** dentro de tu código para poder tomar decisiones dentro del mismo, siendo este una alternativa al condicional Switch.

Arreglos o arrays en Java.

En Java, los **arrays** (o **arreglos**) son estructuras de datos que permiten almacenar múltiples valores del mismo tipo en una única variable. Un array tiene un tamaño fijo que se define en el momento de su creación, y cada elemento dentro del array se accede a través de un índice numérico, que comienza desde 0.

Para declarar un array, es necesario especificar el tipo de datos que contendrá (**como int, String, etc.**) seguido de corchetes **[]**. Los **arrays** permiten almacenar datos de manera estructurada y acceder a ellos de forma eficiente mediante bucles o de manera directa utilizando el índice. Además, Java proporciona métodos y funciones para realizar operaciones comunes con **arrays**, como ordenarlos, buscar elementos o calcular su tamaño usando **array.length**.

Los **arrays** son fundamentales en la programación, ya que permiten manejar grandes cantidades de datos de forma organizada y eficiente, siendo una de las estructuras de datos más utilizadas para almacenar y manipular colecciones de valores.

Los arreglos guardan la información en espacios ordenados en números del 0 al tamaño que el asignemos por ejemplo si queremos un arreglo que guarde 5 números entonces los espacios ocupados serán del 0 al 4.

**String [] Arreglo = { 1 ,2 ,3 ,4 ,5}
0 ,1 ,2 ,3 ,4**

Esos serían los espacios que ocuparía en memoria empezando desde el cero hasta la cantidad de datos que asignemos. De esa manera podemos llamar los datos de un arreglo con diferentes funciones y bucles.

Te mostrare un ejemplo para recorrer arreglos de forma fácil y sencilla.

Primero declaramos una variable que almacenara un arreglo.

```
// Declarar y asignar valores al array  
int[] numeros = {10, 20, 30, 40, 50};
```

Luego declaramos un ciclo for para poder correr ese arreglo de la siguiente forma:

```
// Recorrer el array e imprimir sus elementos  
for (int i = 0; i < numeros.length; i++) {  
    System.out.println("Elemento en el índice " + i + ": " + numeros[i]);  
}
```

De esta manera podemos recorrer un arreglo de manera fácil y sencilla.

Listas.

En Java, una lista es una colección que permite almacenar elementos de manera ordenada y accesible. A diferencia de los **arrays**, las listas pueden crecer o reducirse dinámicamente, lo que las convierte en una opción flexible para gestionar conjuntos de datos. La interfaz más comúnmente utilizada para trabajar con listas en Java es List, que forma parte del marco de colecciones (**Java Collections Framework**).

Las implementaciones más populares de la interfaz **List** son **ArrayList** y **LinkedList**. **ArrayList** ofrece un acceso rápido a los elementos mediante índices, pero su rendimiento puede verse afectado al realizar inserciones o eliminaciones en el medio de la lista. Por otro lado, **LinkedList** es más eficiente para estas operaciones, ya que utiliza nodos enlazados, pero su acceso a elementos por índice es más lento.

Las listas permiten la duplicación de elementos y ofrecen métodos útiles para manipular los datos, como agregar, eliminar, buscar y ordenar elementos. Su versatilidad y facilidad de uso hacen que sean una de las estructuras de datos más empleadas en Java para gestionar colecciones de objetos de manera eficiente y ordenada.

Para crear una lista te lo mostrare en un ejemplo práctico para que puedas entender de una mejor manera:

Primero definimos la lista utilizando **ArrayList** y especificando su tipo de dato.

```
// Crear una lista de nombres  
ArrayList<String> nombres = new ArrayList<>();
```

Al colocar el nombre de la lista creamos un nuevo objeto de tipo ArrayList, esto nos pedirá que importemos su respectiva biblioteca entonces la importas.

La lista estará vacía ya que no hay ningún elemento, para poder ingresar elementos a la lista usamos el método **add** para agregar contenido a nuestra lista.

```
// Agregar elementos a la lista  
nombres.add("Juan");  
nombres.add("María");  
nombres.add("Pedro");  
nombres.add("Ana");
```

Luego de declarar el contenido de la lista procedemos a mostrarlo esto se logra con un ciclo de cualquier tipo, como el siguiente.

```
// Imprimir los nombres en la lista  
System.out.println("Lista de nombres:");  
for (String nombre : nombres) {  
    System.out.println(nombre);  
}
```

Esto la ira imprimiendo poco a poco y mostrándolos en consola. También podemos acceder a elementos individualmente de la siguiente manera:

```
// Acceder a un elemento específico  
String primerNombre = nombres.get(0);  
System.out.println("El primer nombre en la lista es: " + primerNombre);
```

Ciclos y Bucles en java.

En Java, los ciclos y bucles son estructuras de control que permiten ejecutar repetidamente un bloque de código mientras se cumpla una condición específica. Estas estructuras son fundamentales para la automatización de tareas y el procesamiento de colecciones de datos, ya que eliminan la necesidad de escribir código repetitivo.

Los tipos principales de bucles en Java son:

1. **for**: Se utiliza cuando se conoce el número de iteraciones de antemano. Este bucle incluye una inicialización, una condición y un incremento o decremento, lo que proporciona un control preciso sobre el número de veces que se ejecuta.
2. **while**: Este bucle ejecuta el bloque de código mientras la condición especificada sea verdadera. Es útil cuando no se conoce de antemano cuántas iteraciones se realizarán.
3. **do-while**: Similar al bucle while, pero garantiza que el bloque de código se ejecute al menos una vez, ya que la condición se evalúa después de la ejecución.

Los bucles son esenciales en la programación, permitiendo realizar tareas repetitivas, recorrer colecciones de datos y aplicar operaciones en cada elemento de manera eficiente. Su uso adecuado contribuye a un código más limpio y fácil de mantener.

Bucle While.

El ciclo **while** en Java es una estructura de control que permite ejecutar un bloque de código de manera repetitiva mientras se cumpla una condición booleana específica. Su sintaxis básica consiste en la palabra clave **while**, seguida de la condición entre paréntesis y un bloque de código entre llaves.

El ciclo evalúa la condición antes de cada iteración; si la condición resulta verdadera (true), se ejecuta el bloque de código. Si es falsa (false), el ciclo termina y el flujo del programa continúa con la siguiente instrucción después del bloque.

Este tipo de bucle es especialmente útil cuando no se conoce de antemano cuántas veces se necesita repetir el bloque de código. Sin embargo, es crucial asegurarse de que la condición eventualmente se vuelva falsa para evitar bucles infinitos, que pueden causar que el programa se congele o se bloquee. El ciclo **while** es una herramienta poderosa para tareas que requieren repetición y control dinámico del flujo de ejecución en un programa.

La estructura básica de un ciclo **While** es la siguiente:

```
while (condicional) {
```

```
Argumento
```

```
}
```


Para entenderlo mejor usaremos un ejemplo práctico. Pensemos que queremos hacer un contador de números desde el número 1 al 5 para crearlo usaremos un ciclo while.

Primero creamos un iterador el cual será una variable numérica que nos servirá como contador, en los ciclos se usa muy a menudo así que es mejor que la tomes en cuenta en cada uno de tus ciclos.

```
int contador = 1; // Inicializamos el contador
```

Luego inicializamos el contador en el ciclo, colocamos el contador en la condicional de nuestro bucle.

```
// Ciclo while que se ejecuta mientras el contador sea menor o igual a 5
while (contador <= 5) {
```

Acá le indicamos que si el contador es menor o igual a 5 este debe ir ejecutando repetidamente el ciclo hasta cumplir la condición, ósea que el contador debe de tener un número mayor a 5. Pero para ello debemos hacer que avance el contador, para ello en el argumento del bucle colocaremos que el contador vaya sumando por cada vuelta que haga el bucle.

```
// Ciclo while que se ejecuta mientras el contador sea menor o igual a 5
while (contador <= 5) {
    System.out.println("Número: " + contador);
    contador++; // Incrementamos el contador
}
```

Con esto podemos hacer un bucle dentro de java sin mayores complicaciones.

Bucle DO-WHILE.

El ciclo do-while en Java es una estructura de control que permite ejecutar un bloque de código al menos una vez y, posteriormente, repetir esa ejecución mientras se cumpla una condición booleana. Su sintaxis se distingue por la disposición de la condición: se evalúa al final del bloque de código, lo que garantiza que el código se ejecute al menos una vez, independientemente de si la condición es verdadera o falsa en la primera evaluación.

La estructura del ciclo do while es la siguiente:

```
do{
    // Código a ejecutar
}while(condición);
```

El ciclo do-while es útil en situaciones donde se requiere que una acción se ejecute al menos una vez, como en la entrada de datos del usuario, donde es necesario mostrar un mensaje o realizar un cálculo antes de decidir si continuar o no. Sin embargo, al igual que con otros ciclos, es importante asegurarse de que la condición eventualmente se vuelva falsa para evitar bucles infinitos.

Veamos un ejemplo práctico para mayor entendimiento.

Primero como en el ejemplo pasado declaramos un iterador como en la siguiente imagen:

```
int contador = 1; // Inicializamos el contador
```

Luego en lugar de iniciar el while iniciamos la sección do y declaramos lo que se ejecutara.

```
// Ciclo do while que se ejecuta al menos una vez  
do {  
    System.out.println("Número: " + contador);  
    contador++; // Incrementamos el contador
```

Posteriormente declaramos el condicional con ayuda del while.

```
    contador++; // Incrementamos el contador  
} while (contador <= 5);
```

Quedando todo de la siguiente manera:

```
int contador = 1; // Inicializamos el contador  
  
// Ciclo do while que se ejecuta al menos una vez  
do {  
    System.out.println("Número: " + contador);  
    contador++; // Incrementamos el contador  
} while (contador <= 5);  
  
}
```

Bucle FOR.

El ciclo **for** en Java es una estructura de control que permite ejecutar un bloque de código un número específico de veces, facilitando la iteración sobre un rango de valores. Su sintaxis se compone de tres partes: la inicialización, la condición y la actualización, todas definidas en la cabecera del ciclo. Esto lo convierte en una opción ideal cuando se conoce de antemano cuántas iteraciones se desean realizar.

La estructura básica del ciclo for podemos observarla en la siguiente imagen:

```
for(inicialización, condición, actualización){  
    argumento  
    }
```

1. **Inicialización:** Se ejecuta una vez al inicio del ciclo, y generalmente se utiliza para definir e inicializar una variable de control.
2. **Condición:** Se evalúa antes de cada iteración; si es verdadera (true), el bloque de código se ejecuta. Si es falsa (false), el ciclo termina.
3. **Actualización:** Se ejecuta al final de cada iteración, y suele usarse para modificar la variable de control (incremento o decremento).

El ciclo for es particularmente útil para recorrer arrays y colecciones, así como para realizar tareas repetitivas donde el número de iteraciones es conocido. Su estructura compacta y clara mejora la legibilidad del código, facilitando su comprensión y mantenimiento.

Ejemplo práctico:

```
// Ciclo for que cuenta desde 1 hasta 5  
for (int i = 1; i <= 5; i++) {  
    System.out.println("Número: " + i);  
}
```

Podemos observar en la imagen que inicializamos el iterador, condición y actualización en el mismo ciclo sin declararla afuera o dentro del ciclo dejando más orden en el código.

Funciones en java.

En Java, una función (también conocida como método) es un bloque de código que realiza una tarea específica y puede ser reutilizado en diferentes partes de un programa. Las funciones permiten dividir el código en unidades modulares, facilitando su organización, mantenimiento y reutilización. Una función puede recibir datos de entrada a través de parámetros y, opcionalmente, devolver un resultado.

La estructura básica de una función en Java incluye:

1. **Modificadores de acceso:** Definen la visibilidad de la función (p. ej., public, private).
2. **Tipo de retorno:** Especifica el tipo de dato que la función devolverá (p. ej., int, String, void si no devuelve nada).
3. **Nombre de la función:** Identifica la función y debe ser descriptivo de su propósito.
4. **Parámetros:** Valores que la función recibe como entrada entre paréntesis, separados por comas si hay más de uno.
5. **Cuerpo de la función:** El bloque de código que se ejecuta cuando la función es invocada.

Estructura de una función (Ejemplo gráfico):

```
public int sumar( "parámetros"){  
  
    argumentos  
}
```

Esta es la estructura base de una función o método dentro de lo que sería Java, lo usamos para modularizar mejor el código y tener más ordenados diferentes funciones que podrían tener nuestras aplicaciones.

Te mostrare un ejemplo en código para mayor comprensión de las funciones:

Primero declaremos dos variables estáticas para que no nos de problemas al declararlas.

```
static int numero1 = 2;  
static int numero2 = 2;
```

Luego declaramos una función llamada sumar, esta también siempre será de tipo estática:

```
public static int sumar(){  
    return numero1+numero2;  
}
```

A esta función le especificamos que el tipo de dato que va a retornar es de tipo entero, al especificarle el tipo de dato nosotros estaremos obligados a retornar un dato de ese tipo.

Luego de especificar la función ahora debemos de mostrarla en pantalla de la siguiente manera.

```
}  
public static void main(String[] args) {  
  
    System.out.println("La suma 2 +2 es igual a: "+ sumar());  
}
```

Con este código podremos mostrar el resultado de la suma llamando el contenido de la función, el contenido es la suma de dos números ósea 2 +2.

Parámetros en funciones.

En Java, los parámetros son valores que se pasan a una función (o método) para que esta realice operaciones con ellos. Los parámetros permiten que las funciones sean más flexibles y reutilizables, ya que el mismo código puede procesar diferentes datos según los valores que se le pasen al ser invocada.

Cuando se define una función, los parámetros se especifican entre paréntesis en la declaración del método, y cada parámetro debe tener un tipo de dato y un nombre que lo identifique. Estos parámetros actúan como variables locales dentro del método, y sus valores son proporcionados cuando se llama a la función, en lo que se conoce como "argumentos".

Practiquémoslo en un ejemplo, tomemos la función anterior y borremos las variables y coloquémosla en el paréntesis de la función.

```
public static int sumar( int numero1, int numero2){
    return numero1+numero2;
}
```

Ahora cada vez que nosotros llamemos a la función debemos de especificarle los datos dentro de los paréntesis de la siguiente forma.

```
public static void main(String[] args) {
    System.out.println("La suma 2 +2 es igual a: "+ sumar(2,2));
}
```

Esto hará que la función reciba dichos datos y ejecute el código que le establecimos en su argumento.

Programación orientada a Objetos.

La Programación Orientada a Objetos (POO) en Java es un paradigma de desarrollo que organiza el software en torno a objetos, en lugar de funciones o procedimientos. Este enfoque se basa en la idea de modelar entidades del mundo real mediante la creación de objetos que poseen atributos y comportamientos. En Java, los objetos son instancias de clases, que actúan como plantillas o modelos para definir las características y capacidades de esos objetos.

Los cuatro pilares fundamentales de la POO en Java son:

1. **Encapsulamiento:** Consiste en restringir el acceso a los detalles internos de un objeto y exponer únicamente lo necesario a través de interfaces controladas. Esto se logra mediante modificadores de acceso como `private`, `protected`, y `public`. El encapsulamiento garantiza que los datos de un objeto estén protegidos y se manipulen solo a través de métodos definidos, lo que mejora la seguridad y la integridad del código.
2. **Herencia:** Es el mecanismo que permite que una clase adquiera las propiedades y comportamientos de otra clase. Esto promueve la reutilización del código y facilita la creación de jerarquías de clases. Las clases "hijas" o "subclases" pueden extender o modificar las funcionalidades de las clases "padre" o "superclases", lo que permite una mayor flexibilidad y escalabilidad en el diseño de sistemas.
3. **Polimorfismo:** Permite que un objeto adopte diferentes formas según el contexto. En la POO, esto significa que los objetos pueden ser tratados como instancias de su propia clase o de cualquiera de sus clases base. El polimorfismo facilita la creación de sistemas más dinámicos y flexibles, ya que permite que diferentes objetos respondan de manera distinta al mismo mensaje o método.

4. **Abstracción:** Es el proceso de ocultar los detalles complejos de la implementación y exponer solo lo esencial. Las clases abstractas y las interfaces son herramientas clave en la abstracción, ya que permiten definir un comportamiento general que será implementado por clases más específicas. La abstracción permite simplificar la interacción entre diferentes componentes del sistema y facilita el desarrollo de software modular y extensible.

Estos pilares trabajan juntos para mejorar la modularidad, la reutilización y la mantenibilidad del código. La POO en Java es ampliamente utilizada en el desarrollo de aplicaciones complejas, ya que proporciona un marco estructurado y lógico para organizar y gestionar el código. Además, permite que los desarrolladores construyan sistemas robustos y escalables, donde los objetos interactúan entre sí de manera coherente y ordenada.

Objetos en POO java.

En la Programación Orientada a Objetos (POO) en Java, un objeto es una instancia de una clase, y representa una entidad que tiene un estado y un comportamiento. El estado de un objeto está definido por sus **atributos** (también conocidos como propiedades o variables de instancia), mientras que su comportamiento está definido por sus **métodos**. Los objetos son las unidades fundamentales en la POO, y se utilizan para modelar entidades del mundo real o conceptos abstractos dentro del software.

Un objeto en Java tiene las siguientes características:

1. **Estado:** Representa los datos que un objeto almacena en sus variables de instancia. Cada objeto puede tener valores diferentes para sus atributos, lo que le otorga un estado único. Por ejemplo, un objeto "Coche" puede tener atributos como "color", "marca" y "modelo", que definen su estado particular.
2. **Comportamiento:** Se refiere a las operaciones o funciones que un objeto puede realizar. Los métodos de la clase definen las acciones que un objeto puede ejecutar, como acelerar, frenar o girar en el caso de un coche. El comportamiento de un objeto está determinado por las interacciones con otros objetos y con su propio estado interno.
3. **Identidad:** Cada objeto tiene una identidad única, incluso si dos objetos tienen el mismo estado y comportamiento. La identidad permite diferenciar un objeto de otro en la memoria. En Java, esto se gestiona mediante referencias, ya que los objetos viven en la memoria y se accede a ellos a través de estas referencias.

En Java, los objetos se crean utilizando la palabra clave **new**, que invoca al **constructor** de una clase. El constructor es un método especial que inicializa el estado del objeto al momento de su creación.

Los objetos son fundamentales en la POO, ya que permiten modelar situaciones y conceptos complejos de manera intuitiva y modular. La interacción entre objetos a través de métodos y mensajes es la esencia de la programación orientada a objetos, y permite que el software sea flexible, reutilizable y fácil de mantener.

Estructura grafica de un objeto en java:

Referencia a la
clase a tratar
como objeto.

Nombre del
objeto.

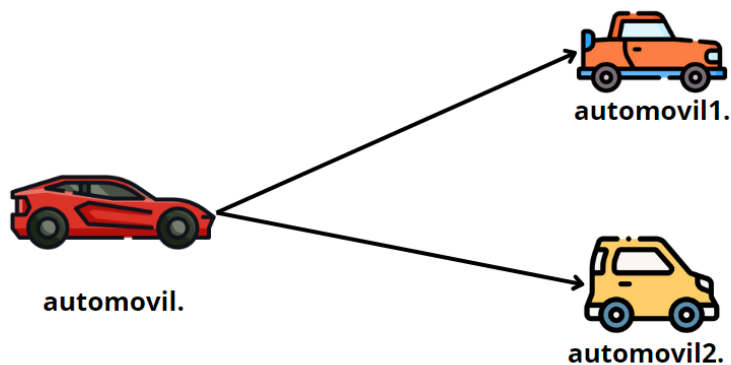
Creación del nuevo
objeto.

Objeto objeto = new Objeto()

Para entender la forma en la que funcionan los objetos podemos tomarlo de la siguiente forma:

Supongamos que tenemos una clase llamada **automóvil** y esta clase tiene atributos y métodos dentro del mismo, estos atributos y métodos se usaran para crear automóviles con diferentes características. Es una ventaja el hecho de tomar los datos de la clase base (**automóvil**) ya que de este modo no creamos los métodos y atributos desde cero.

Esquema del ejemplo:



Automóvil representaría la clase base ya que este tiene todos los métodos y atributos como, por ejemplo:

Atributos:

- Color.
- Marca.
- Kilometraje.
- Numero de neumáticos.
- Cantidad de luces.

Métodos:

- Arrancar.
- Frenar
- Acelerar
- Encender y apagar luces.
- Derrapar.

Tomamos los atributos como características del objeto y los métodos como las acciones del mismo, en el esquema podremos ver que, al heredar los datos, estos no deben ser vueltos a declarar en los automóviles 1 y 2. Veamos un ejemplo en código para entenderlo mejor.

Tenemos una clase aparte de la clase principal llamada **Carro**, las clases que representaran un objeto siempre deben iniciar con mayúsculas. Esta clase tiene métodos y atributos.

```
public class Carro {  
    public String Color;  
    public String Marca;  
    public String Kilometraje;  
    public int CantidadNeumaticos;  
    public int CantidadLuces;  
  
    public String arrancar(){  
  
        return "El automovil esta arrancando";  
    }  
  
    public String informacion(String Marca, String Color, String Kilometraje, int CantidadNeumaticos, int CantidadLuces){  
  
        this.Marca = Marca;  
        this.Kilometraje = Kilometraje;  
        this.Color = Color;  
        this.CantidadNeumaticos = CantidadNeumaticos;  
        this.CantidadLuces = CantidadLuces;  
        return "Informacion del automovil: \n " + "Marca: " + Marca + "\n Color: " + Color + "\n Kilometraje: " + Kilometraje +  
            "\n Cantidad de neumaticos: " + CantidadNeumaticos + "\n Cantidad de luces: " + CantidadLuces;  
    }  
}
```

Nosotros mediante un **objeto** podemos acceder a estos métodos y modificar cada uno de estos, como podemos ver en la imagen anterior tenemos diferentes métodos y en uno de estos pedimos parámetros ya que será un método que pedirá información sobre el automóvil.

Una vez tenemos la clase base procedemos a acceder a ella creando un **objeto** desde nuestra clase principal.

```
public static void main(String[] args) {  
  
    Carro automovil1 = new Carro();  
}
```

Creamos un objeto de la clase **Carro** y lo llamaremos **automovil1** y será igual a un objeto nuevo.

Con este objeto accederemos a todas las variables de nuestra clase **Carro** y editarlas sin ningún problema como lo hacemos en la siguiente imagen:

```
public static void main(String[] args) {  
    Carro automovil1 = new Carro();  
    String metodo = automovil1.arrancar();  
    System.out.println("Informacion del automovil: \n" + automovil1.informacion("Toyota", "Azul", "3000KM", 4, 6));  
    System.out.println("Mostrando metodo: " + metodo);  
}
```

Aquí guardamos el método en una variable (opcional) o simplemente lo mandamos a llamar en consola directamente a su vez también llamamos el método de información e incluimos los datos nuevos que queremos establecer según el modelo del objeto, para acceder a todos los métodos y atributos siempre usamos un punto para referenciarlos

Ejemplo: automovil1.metodo1

De esta manera podemos acceder a dichos atributos y métodos y modificarlos a nuestra conveniencia.

Ejercicios de práctica.

¡Ya has llegado hasta esta parte del libro! ¡Déjame felicitarte! Ahora puedes practicar con 10 diferentes ejercicios para poner en práctica lo aprendido, a su vez también puedes desarrollar tu lógica de programación.

Estos ejercicios puedes aplicarle cualquier solución que se te ocurra, pero siempre completando el ejercicio y llegando al resultado esperado.

1. Calculadora básica (if-else y switch)

- Crea una aplicación de consola que solicite dos números y luego pregunte qué operación realizar (+, -, *, /). Utiliza una estructura if-else o switch para realizar la operación correspondiente y mostrar el resultado.
-

2. Determinar si un número es par o impar

- Escribe un programa que pida un número al usuario y determine si es par o impar. Utiliza la estructura if-else para hacer la verificación.
-

3. Sumar elementos de un array

- Crea un array de enteros con 5 elementos. Usa un ciclo for para sumar todos los elementos del array e imprime el resultado.
-

4. Invertir una cadena

- Escribe un programa que tome una cadena ingresada por el usuario y la invierta. Por ejemplo, si el usuario ingresa "Java", el programa debe mostrar "avaJ".
-

5. Crear una clase Círculo

- Define una clase Círculo que tenga como atributo el radio. La clase debe tener un método que calcule el área y otro que calcule el perímetro. Crea un objeto de esta clase en el main y muestra el área y el perímetro.
-

6. Encontrar el número mayor en un array

- Escribe un programa que tome un array de números enteros y encuentre el número mayor. Usa un ciclo for para recorrer el array y comparar los elementos.
-

7. Uso de la clase ArrayList

- Crea un programa que use la clase ArrayList para almacenar los nombres de los estudiantes de una clase. Permite agregar nombres y luego imprimir todos los nombres almacenados.
-

8. Contar vocales en una cadena

- Escribe una función que tome una cadena como parámetro y devuelva el número de vocales que contiene (a, e, i, o, u). Haz que el programa cuente tanto vocales mayúsculas como minúsculas.
-

9. Juego de adivinanza

- Crea un juego donde el programa genere un número aleatorio entre 1 y 100. El usuario debe intentar adivinarlo. Por cada intento, el programa le dice si el número es mayor o menor que su suposición. El juego termina cuando el usuario adivina el número.
-

10. Tablas de multiplicar

- Escribe un programa que pida al usuario un número y luego imprima la tabla de multiplicar de ese número desde el 1 hasta el 10.

Próximos pasos para convertirte en un desarrollador profesional.

hora que has aprendido los fundamentos de Java, aquí hay algunas sugerencias para continuar tu viaje de programación:

1. **Desarrollar proyectos personales:** Intenta crear aplicaciones pequeñas que te interesen, como un gestor de tareas, un juego simple o una aplicación de calculadora. Esto te ayudará a aplicar lo que has aprendido.
2. **Contribuir a proyectos de código abierto:** Participar en proyectos de código abierto en plataformas como GitHub te dará experiencia práctica y te permitirá trabajar en colaboración con otros desarrolladores.
3. **Aprender sobre frameworks y bibliotecas:** Explora frameworks populares como Spring o JavaFX para desarrollar aplicaciones web y de escritorio, respectivamente. Familiarizarte con estas herramientas ampliará tus habilidades. ¡Mas adelante encontraras recursos para desarrollar tus habilidades al máximo!
4. **Estudiar patrones de diseño:** Investiga sobre patrones de diseño y arquitecturas de software para mejorar la calidad y mantenibilidad de tu código.
5. **Explorar otros lenguajes de programación:** Aprender un nuevo lenguaje te proporcionará diferentes perspectivas sobre la programación y enriquecerá tu conjunto de habilidades. En la próxima página encontraras diferentes rutas de aprendizajes para expandir tu conocimiento en programación.

Recursos adicionales.

Aprende más sobre Java:



Vinculo del curso de java.

Sigue aprendiendo:



<https://www.achirou.com/>